COLLEGE PROJECT DOCUMENTATION

LEGALEASE

AI-Powered Legal Document Generator

A project documentation prepared from the supplied LegalEase source files

Project Title	LegalEase – AI-Powered Legal Document Generator
Technology	Python, Streamlit, FastAPI, Google Gemini
Application Type	Web-based AI-assisted document drafting application
Project Version	1.0.0 (as specified in the supplied FastAPI application)
Student Name	_____
Register Number	_____
Department	_____
College	_____
Academic Year	_____

Submitted in partial fulfilment of the requirements for the relevant academic programme.

CERTIFICATE

This is to certify that the project titled **“LegalEase – AI-Powered Legal Document Generator”** is a project documentation prepared based on the software source files, README material, requirements file, and test files supplied for this project.

Project Guide	_____
Head of the Department	_____
Internal Examiner	_____
External Examiner	_____
Date	_____

DECLARATION

I hereby declare that this project documentation describes the supplied LegalEase application and its documented implementation components. The report is organized for academic presentation and is based on the project files provided for preparation of this document.

Place: _____

Date: _____

Signature of the Student: _____

TABLE OF CONTENTS

1	Abstract
2	Introduction
3	Problem Statement
4	Objectives
5	Scope of the Project
6	Technology Stack
7	System Requirements
8	System Architecture and Workflow
9	Functional Modules
10	User Interface and Application Flow
11	Backend API Design
12	Data Models
13	Document Export and Formatting
14	Project Structure
15	Testing and Validation
16	Error Handling
17	Security and Responsible Use
18	Limitations
19	Future Enhancements
20	Conclusion
21	References and Source Files
22	Appendix: File-wise Documentation

1. ABSTRACT

LegalEase is an AI-powered application designed to help users create structured legal-document drafts. The supplied project combines a Streamlit-based user interface with a FastAPI backend and an AI generation layer identified in the project documentation as Google Gemini. Users select a document type, enter parties and terms, provide an effective date and optional jurisdiction, and submit the information for document generation.

The generated draft is displayed in an editable preview area and can be exported in TXT, DOCX, or PDF formats. The backend exposes a document-generation endpoint and health-check endpoints. The project also contains automated tests for the root and health endpoints. The supplied README identifies optional logo branding, API documentation, and a modular backend/frontend structure.

Important academic note: this documentation describes the supplied files as they were provided. Some implementation files named in the README (such as the document-generation service and Gemini generator) were listed in the project structure but were not included among the supplied source files; therefore their internal implementation is not described beyond what the supplied materials state.

2. INTRODUCTION

Preparing structured legal documents manually can require repeated formatting and careful organization of information. LegalEase proposes an AI-assisted workflow in which users provide structured inputs and receive a generated legal-document draft that can be reviewed, edited, and exported.

The frontend is implemented using Streamlit. It provides document-type selection, input fields for parties and terms, an effective date selector, optional jurisdiction, additional instructions, generation feedback, editable output, and download controls. The backend is implemented using FastAPI and exposes a POST /generate endpoint as well as root and health endpoints.

The project documentation identifies Google Gemini as the AI integration and lists Python, python-docx and fpdf2 among the main technologies. The application therefore demonstrates an end-to-end AI application pattern: user input → API request → AI-assisted generation → editable result → document export.

3. PROBLEM STATEMENT

Users may need structured document drafts without manually creating the entire document layout from scratch. The project addresses this by accepting essential document information through a web interface and sending it to a backend generation service. The resulting content can then be reviewed and exported.

4. OBJECTIVES

• Provide a simple interface for entering legal-document requirements.

• Support multiple document categories such as Rental Agreement, Employment Agreement, Non-Disclosure Agreement, Service Agreement, Affidavit, Loan Agreement and Partnership Agreement.

• Generate structured document drafts using the configured AI generation service.

• Allow users to edit the generated draft before export.

• Provide TXT, DOCX and PDF export options.

• Provide a REST API for document generation.

• Provide basic service and health-check endpoints for validation.

5. SCOPE OF THE PROJECT

• AI-assisted drafting of structured legal-document content.

• Web-based interaction through Streamlit.

• Backend processing through FastAPI.

• Document request validation through Pydantic models.

• Export of generated text to TXT, DOCX and PDF.

• Optional logo support as documented by the project README.

• Basic API health monitoring and automated endpoint tests.

6. TECHNOLOGY STACK

Technology / Package	Role in the Project
Python	Primary programming language.
Streamlit	Frontend/web interface for collecting inputs and displaying generated documents.
FastAPI	Backend framework and REST API layer.
Google Gemini / google-genai	AI generation integration identified by the project documentation.
Pydantic	Validation and definition of document request/response models.
python-docx	Creation of DOCX document output.
fpdf2	Creation of PDF document output.
Pillow	Listed dependency; relevant to image/logo handling.
Requests	Frontend-to-backend HTTP communication.
python-dotenv	Loading environment variables.
Uvicorn	ASGI server listed in the requirements.
Pytest	Testing framework used by the supplied test file.

7. SYSTEM REQUIREMENTS

Software requirements:

• Python environment capable of installing the packages listed in requirements.txt.

• FastAPI and Uvicorn for the backend service.

• Streamlit for the frontend.

• Configured environment variables for the backend/AI service as required by the project setup.

• A modern web browser for accessing the Streamlit interface.

Hardware requirements:

• A computer capable of running Python, the backend, and the Streamlit application.

• Internet/network access may be required for configured AI/API services.

The supplied requirements file lists the principal Python dependencies, including FastAPI, Uvicorn, Streamlit, python-docx, fpdf2, Pillow, Requests, python-dotenv, Pydantic, google-genai and Pytest.

8. SYSTEM ARCHITECTURE AND WORKFLOW

The supplied project follows a frontend–backend architecture. The Streamlit application collects the user's inputs and sends a JSON request to the FastAPI backend. The backend validates the request, invokes the document-generation service, and returns generated content and model information. The frontend then displays the result and provides export controls.

Layer	Component	Responsibility
Presentation	Streamlit app	Collects document inputs, displays status messages, shows editable output, and prov
API	FastAPI	Receives document-generation requests and exposes service endpoints.
Validation	Pydantic schemas	Validates required fields and defines the response structure.
Generation	Document service / Gemini integration	Generates document content using the configured AI generation layer; internal files w
Export	TXT / DOCX / PDF helpers	Converts the generated text into downloadable formats.
Testing	Pytest / TestClient	Validates root and health endpoint behavior.

Workflow

1. User selects a document type. → 2. User enters parties and terms. → 3. User selects the effective date and optionally enters jurisdiction/instructions. → 4. Frontend validates required fields. → 5. Frontend sends a POST request to /generate. → 6. Backend validates the request and calls the generation service. → 7. Generated content is returned. → 8. User edits the draft if necessary. → 9. User downloads TXT, DOCX, or PDF.

The frontend uses the backend URL from the BACKEND_URL environment variable, defaulting to http://127.0.0.1:8000 when it is not otherwise configured.

9. FUNCTIONAL MODULES

9.1 Frontend Module

The Streamlit frontend presents the LegalEase title and describes the application as an AI-powered legal document draft generator. It provides a sidebar with project information and a warning that AI-generated legal documents may contain errors and should be reviewed by a qualified legal professional before signing or relying on them.

9.2 Document Input Module

The interface supports the following predefined document types: Rental Agreement, Employment Agreement, Non-Disclosure Agreement, Service Agreement, Leave and License Agreement, Affidavit, Loan Agreement, Partnership Agreement, and Custom. For Custom, the user can enter a document type manually.

Required inputs are document type, parties, and terms and conditions. Effective date is also collected. Jurisdiction and additional instructions are optional.

9.3 Generation Module

When the Generate Document button is selected and required inputs are present, the frontend creates a JSON payload containing document_type, parties, terms, effective_date, jurisdiction, and additional_instructions. It sends this payload to the backend /generate endpoint.

9.4 Editing Module

After successful generation, the returned content is displayed in an editable text area. The edited text becomes the content used for export.

9.5 Export Module

The interface provides three download controls: TXT, DOCX, and PDF. The supplied exporter implementation uses UTF-8 bytes for TXT, Times New Roman styling for DOCX, and FPDF for PDF generation.

10. USER INTERFACE AND APPLICATION FLOW

Screen Area	Function
Title area	Displays LegalEase and the AI-powered legal document draft generator description.
Sidebar	Shows project description, supported exports, responsible-use warning, and backend URL.
Document Type	Selects a predefined document type or allows a Custom type.
Parties	Collects party information such as landlord/tenant in the supplied example.
Terms and Conditions	Collects rent, duration, duties, payment terms, or other document-specific terms.
Effective Date	Collects the date on which the document becomes effective.
Jurisdiction	Optional location/jurisdiction input.
Additional Instructions	Optional instructions for generation.
Generated Document	Shows the returned draft in an editable text area.
Download	Exports the final edited content as TXT, DOCX or PDF.

11. BACKEND API DESIGN

The FastAPI application is created with the title “LegalEase API” and version “1.0.0”. CORS middleware is configured for the local Streamlit origins listed in the supplied main.py.

Method	Endpoint	Purpose	Response / Error behavior
GET	/	Service root/status endpoint.	Returns service name, running status, and /docs path.
GET	/health	Health check.	Returns status: ok.
POST	/generate	Generate a document draft.	Returns document type, generated content, and model; maps runtin

The /generate route accepts a validated DocumentRequest, calls the document-generation service, and returns a DocumentResponse. Runtime errors are mapped to HTTP 503, while other generation failures are mapped to HTTP 502.

12. DATA MODELS

The supplied schemas.py defines two Pydantic models.

Model	Field	Description
DocumentRequest	document_type	Required document category/name.
DocumentRequest	parties	Required party information.
DocumentRequest	terms	Required terms and conditions.
DocumentRequest	effective_date	Required effective date string.
DocumentRequest	jurisdiction	Optional jurisdiction; defaults to empty string.
DocumentRequest	additional_instructions	Optional instructions; defaults to empty string.
DocumentResponse	document_type	Document category/name returned by the API.
DocumentResponse	content	Generated document content.
DocumentResponse	model	Model identifier returned by the generation service.

13. DOCUMENT EXPORT AND FORMATTING

The supplied exporter implementation contains three conversion functions: create_txt_bytes, create_docx_bytes, and create_pdf_bytes.

TXT Export

The TXT function converts the document content into UTF-8 encoded bytes.

DOCX Export

The DOCX function creates a Word document, sets the normal style to Times New Roman at 11 points, adds the document type as a bold 16-point title, and writes the generated content line by line.

PDF Export

The PDF function uses FPDF, sets automatic page breaks and margins, adds the document type as a 16-point title, and writes the content using an 11-point Helvetica font. The supplied implementation replaces unsupported Unicode characters before writing them to the PDF.

The README additionally documents optional logo branding: if a project/company logo is available, a logo.png file can be placed in the assets location and the application can use it in DOCX and PDF documents. The supplied README notes that the project also works without a logo.

14. PROJECT STRUCTURE

The README describes the project as a modular structure containing backend, frontend, utility, asset and test areas.

```
LegalEase/
■■■ backend/
■   ■■■ __init__.py
■   ■■■ main.py
■   ■■■ routes.py
■   ■■■ schemas.py
■   ■■■ services/
■       ■■■ __init__.py
■       ■■■ document_service.py
■       ■■■ gemini_generator.py
■■■ frontend/
■   ■■■ __init__.py
■   ■■■ app.py
■■■ utils/
■   ■■■ __init__.py
■   ■■■ exporters.py
■   ■■■ text_utils.py
■■■ assets/
■   ■■■ README.txt
■■■ tests/
■   ■■■ __init__.py
■   ■■■ test_health.py
■   ■■■ test_exports.py
■■■ .env
■■■ .env.example
■■■ .gitignore
■■■ requirements.txt
■■■ README.md
```

The supplied files include app.py, main.py, routes.py, schemas.py, README.md, requirements.txt, test_health.py, test_exports.py and __init__.py. The README lists additional service and utility files that were not supplied in the current set of files.

15. TESTING AND VALIDATION

The supplied test_health.py uses FastAPI TestClient to test the application. The root endpoint is expected to return HTTP 200 and a JSON status of “running”. The /health endpoint is expected to return HTTP 200 and a JSON status of “ok”.

Test Case	Input / Action	Expected Result
TC-01	GET /	HTTP 200; status is running.
TC-02	GET /health	HTTP 200; status is ok.
TC-03	Generate without document type	Frontend displays required-field error.
TC-04	Generate without parties	Frontend displays required-field error.
TC-05	Generate without terms	Frontend displays required-field error.
TC-06	Backend connection unavailable	Frontend displays backend connection error.
TC-07	Generation timeout	Frontend displays timeout error.
TC-08	Successful generation	Generated content is displayed and becomes downloadable.

The test_exports.py supplied with the project contains implementations for TXT, DOCX and PDF conversion. The filename/content relationship should be checked against the README structure during

final project integration, because the frontend imports these functions from utils.exporters.

16. ERROR HANDLING

The frontend checks required fields before sending a request. During backend communication it separately handles connection errors, request timeouts, HTTP backend errors, and unexpected frontend exceptions. The backend route handles RuntimeError with HTTP 503 and other exceptions with HTTP 502.

Condition	Handling
Missing document type	Frontend validation message.
Missing parties	Frontend validation message.
Missing terms	Frontend validation message.
Backend unavailable	Connection error message asking the backend to be started.
Request timeout	Timeout message asking the user to retry.
Backend generation error	Backend error detail displayed to the user.

17. SECURITY AND RESPONSIBLE USE

The application documentation itself warns that AI-generated legal documents may contain errors and recommends review by a qualified legal professional before signing or relying on a generated document. This is an important design consideration because the application generates drafts rather than establishing that a document is legally valid for a particular situation.

The backend uses CORS configuration for the two local Streamlit origins shown in main.py. Environment configuration is used for the backend URL, and the requirements include python-dotenv and google-genai. The supplied materials do not provide enough detail to document the internal handling of API keys or the exact AI prompt/generation implementation, because the referenced generation-service files were not included.

18. LIMITATIONS

• The supplied source set does not include the internal document-generation service and Gemini generator files listed in the README.

• AI-generated content may contain errors and therefore requires human review.

• The supplied PDF exporter replaces unsupported Unicode characters, which may affect some non-Latin text in exported PDFs.

• The project structure documented in README and the actual supplied filenames should be reconciled before final deployment.

• The supplied material does not demonstrate production deployment, authentication, database storage, or user account management.

19. FUTURE ENHANCEMENTS

• Add user authentication and role-based access where required.

• Add persistent storage for document drafts and revision history.

- Improve PDF Unicode/font support for multilingual documents.
- Add richer document templates and domain-specific validation.
- Add stronger automated tests for document generation, API validation and all export formats.
- Provide deployment configuration for a hosted environment.
- Add a dedicated preview layout for the final document before download.

20. CONCLUSION

LegalEase demonstrates a modular AI-assisted document-generation workflow using a Streamlit frontend, FastAPI backend, Pydantic data models, and document-export utilities. The supplied project supports several legal-document categories, accepts structured user inputs, sends generation requests to the backend, presents an editable draft, and provides TXT, DOCX and PDF downloads.

The project also includes basic health endpoints and automated tests, making it suitable as an academic demonstration of frontend–backend integration with an AI generation component. For real-world use, the generated content should be treated as a draft and reviewed appropriately, consistent with the warning included in the application.

21. REFERENCES AND SOURCE FILES

This documentation was prepared from the project materials supplied in the conversation:

Source	Use in Documentation
README.md	Project description, technology stack, features, structure, and logo information.
app.py	Frontend interface, input validation, backend communication, editing and download workflow.
main.py	FastAPI application configuration, CORS, root and health endpoints.
routes.py	Document-generation API route and error mapping.
schemas.py	Document request and response data models.
test_health.py	Root and health endpoint tests.
test_exports.py	TXT, DOCX and PDF export implementation supplied in the uploaded file.
requirements.txt	Python package dependencies.
README-1.md	Pytest cache information; not a functional application module.

22. APPENDIX: FILE-WISE DOCUMENTATION

File	Documented Role
app.py	Streamlit frontend. Handles document selection, input collection, generation request, generated-content ec
main.py	Creates the FastAPI application, configures CORS, includes routes, and defines / and /health.
routes.py	Defines POST /generate and converts generation failures into HTTP responses.
schemas.py	Defines DocumentRequest and DocumentResponse using Pydantic.
test_health.py	Uses FastAPI TestClient to validate root and health endpoint responses.
test_exports.py	Contains export helper implementations for TXT, DOCX and PDF in the supplied source set.
requirements.txt	Lists FastAPI, Uvicorn, Streamlit, python-docx, fpdf2, Pillow, Requests, python-dotenv, Pydantic, google-ge
README.md	Provides project overview, features, technology list, structure, and logo instructions.
README-1.md	Describes pytest cache data and states it should not be committed to version control.
__init__.py	Supplied as an empty package initialization file.

Key source-derived code references

```
FastAPI root response:
{
    "service": "LegalEase API",
    "status": "running",
    "docs": "/docs"
}

Health response:
{
    "status": "ok"
}

Document request fields:
document_type
parties
terms
effective_date
jurisdiction
additional_instructions

Document response fields:
document_type
content
model
```

End of Documentation